

Algorithmic Problem Solving 2026

Project Report — APS 2026

12.05.2026

Group H — Karl Thedor Ruby, Kristoffer Mejborn Eliasson, Lukas Shaghashvili-Johannessen
Algorithmic Problem Solving (APS), ITU

Contents

1. Abstract	1
2. Project Metadata	1
3. Contents	1
4. 1 Designed Problem	1
4.1. 1.1 Problem description	1
4.2. 1.2 Intended solution	1
4.3. 1.3 Test-case design	2
4.4. 1.4 Accepted and wrong solutions	2
4.5. 1.5 Input validation & formatting	2
4.6. 1.6 Verify notes	2
5. 2 Source for Designed Problem	2
6. 3 Solved Problem(s)	2
6.1. 3.N Problem metadata	2
6.2. 3.N.1 Solution overview	2
6.3. 3.N.2 Correctness argument	3
6.4. 3.N.3 Complexity and time-limit reasoning	3
6.5. 3.N.4 Empirical worst-case testing	3
6.6. 3.N.5 Implementation notes	3
7. 4 Testing and Validation	3
8. 5 Packaging and Submission	3
9. 6 FAQ / Notes from supervision	4
10. Appendix A — Checklist	4
11. Appendix B — Code listings	4

1. Abstract

Provide a short summary (max 200 words) describing the designed problem(s), the solved problem(s), and the main algorithmic techniques used.

2. Project Metadata

- **Group:** Group H
- **Members:** Karl Thedor Ruby (krub@itu.dk); Kristoffer Mejbørn Eliasson (krme@itu.dk); Lukas Shaghashvili-Johannessen (lush@itu.dk)
- **Supervisor:** [Supervisor name]
- **Course:** Algorithmic Problem Solving (APS 2026)
- **Hand-in:** PDF + source zip
- **Date:** [YYYY-MM-DD]

3. Contents

toc()

4. 1 Designed Problem

Provide a self-contained problem that can be used for automated judging. Make sure `verifyproblem` accepts your files.

4.1. 1.1 Problem description

Give the problem statement as you would present it to contestants. Include input and output formats, constraints, and examples.

4.2. 1.2 Intended solution

Describe the algorithmic idea behind an intended (accepted) solution. Use pseudocode and a short complexity analysis (time and memory). Example:

```
function solve(input):  
    parse input  
    // algorithmic steps  
    return output
```

- **Time complexity:** $O(\dots)$
- **Memory:** $O(\dots)$

4.3. 1.3 Test-case design

Explain how you generated sample and secret inputs. For each test group, state the purpose, parameter ranges, and expected corner cases.

- **Sample tests:** small cases for manual checking.
- **Secret tests:** randomized and worst-case instances to exercise complexity.

4.4. 1.4 Accepted and wrong solutions

Include at least one accepted solution and one or more wrong/inefficient variants. For each wrong solution, explain why it fails (e.g., incorrect logic, TLE, memory).

4.5. 1.5 Input validation & formatting

Specify how your judge should treat malformed input and which assumptions contestants may rely on.

4.6. 1.6 Verify notes

Confirm that `verifyproblem` runs cleanly. If it produced warnings, document them and the fixes applied.

5. 2 Source for Designed Problem

Provide a compact description of the repository layout and how to run the judge and tests locally.

- **Files to include in zip:** problem statement, solutions, test generator(s), `verifyproblem` configuration, README.
- **Run locally:** provide exact commands used to generate and run tests.

6. 3 Solved Problem(s)

For each problem you solved (one per group member as a rule of thumb), include the following subsections.

6.1. 3.N Problem metadata

- **Problem name / Kattis link:** [URL]
- **Short formal I/O description:** (not a repeat of the original prose)

6.2. 3.N.1 Solution overview

Explain the chosen algorithm, key data structures, and why this approach was selected.

6.3. 3.N.2 Correctness argument

Sketch a correctness proof or invariants that guarantee the solution works for all valid inputs.

6.4. 3.N.3 Complexity and time-limit reasoning

Provide asymptotic runtime and memory usage. Discuss how your implementation meets the problem limits and expected worst-case inputs.

6.5. 3.N.4 Empirical worst-case testing

If possible, generate or describe inputs that approximate worst-case behaviour, measure running time, and report results.

6.6. 3.N.5 Implementation notes

- **Language used:** e.g. C++17, Python 3.10
- **Files:** path/to/solution
- **How to run:** ./run_solution input.txt

7. 4 Testing and Validation

- Describe your testing strategy: unit tests, random testing, stress testing, boundaries.
- Provide commands to reproduce tests.

Example commands:

```
# generate tests
python3 tests/generate.py --cases 100 --max-n 1000 > secret.in

# run judge
python3 judge/run_judge.py problems/yourproblem secret.in
```

8. 5 Packaging and Submission

- Ensure the hand-in contains one PDF and one ZIP with all source files.
- ZIP layout recommendation:
 - README.md (how to build/run)
 - problem/ (statement + verifyproblem files)
 - solutions/ (accepted + wrong solutions)
 - tests/ (generators and sample/secret inputs)

9. 6 FAQ / Notes from supervision

Add any course-specific clarifications and the answers to frequently asked questions relevant to your group.

10. Appendix A — Checklist

- ☐ Problem description included
- ☐ Intended solution described
- ☐ Test-case design documented
- ☐ `verifyproblem` passes
- ☐ Source zip prepared
- ☐ Solved problems documented (one per member)

11. Appendix B — Code listings

Include important code snippets or point to files under `solutions/` and `tests/`.

—

If you want, I can also:

- Fill the template with your group metadata and an example problem, or
- Add a `typst` macro to render checkboxes more nicely, or
- Generate a minimal `README.md` and `verifyproblem` example.